

PAPER • OPEN ACCESS

# Solving equations approximately with Grover's search algorithm

To cite this article: Jed Brody and Wenye Song 2025 *Eur. J. Phys.* **46** 065807

View the [article online](#) for updates and enhancements.

## You may also like

- [Explaining Grover's algorithm with a colony of ants: a pedagogical model for making quantum technology comprehensible](#)  
Merel A Schalkers, Kamiel Dankers, Michael Wimmer et al.
- [Periodicity of bipartite walk on biregular graphs with conditional spectra](#)  
Qiuting Chen
- [Designing quantum circuits for decoding binary linear codes](#)  
Hyungkee Yoo, Whan-Hyuk Choi and Yoonjin Lee

# Solving equations approximately with Grover's search algorithm

Jed Brody<sup>1,\*</sup>  and Wenye Song<sup>2</sup>

<sup>1</sup> Department of Physics, Emory University, Atlanta, GA 30322, United States of America

<sup>2</sup> Department of Computer Science, Emory University, Atlanta, GA 30322, United States of America

E-mail: [jbrody@emory.edu](mailto:jbrody@emory.edu)

Received 25 April 2025, revised 12 September 2025

Accepted for publication 3 October 2025

Published 6 November 2025



CrossMark

## Abstract

To enrich curricular materials for quantum computing education, we show how Grover's algorithm can be used to find approximate solutions to algebraic and transcendental equations. This significantly expands the range of exercises based on Grover's algorithm, which typically solves problems with exact binary solutions. These exercises teach and unify a diverse skillset, including binary representation of fractional and negative numbers, implementation of Boolean operations with quantum gates, and comprehensive design of quantum circuits to implement Grover's algorithm.

Supplementary material for this article is available [online](#)

Keywords: quantum computing, Grover's algorithm, numerical methods

## 1. Introduction

Quantum computing is a potentially transformative technology that has sparked widespread interest among educators [1], students [2], and the general public [3, 4]. Universities are developing courses, concentrations, and degrees in quantum information science [5]. We seek to enrich curricular materials with an exercise that can be used in a quantum computing

\* Author to whom any correspondence should be addressed.



Original content from this work may be used under the terms of the Creative Commons Attribution 4.0 licence. Any further distribution of this work must maintain attribution to the author(s) and the title of the work, journal citation and DOI.

course, or as a special topic in undergraduate courses in quantum mechanics, digital electronics, or numerical methods.

The author of a popular textbook on quantum mechanics wrote, ‘There is no general consensus as to...how [quantum mechanics] should be taught’. [6] This is even truer of the newer and rapidly evolving field of quantum computing. Developing new curricular materials, such as those presented here, can only assist in the effort to provide more options and clarity for instructors and students.

Grover’s search algorithm is one of the ‘killer apps’ [7] of quantum computing. Textbooks explain it at an abstract, mathematical level, but there are many applications that can be explored. Besides searching a quantum database [8], Grover’s algorithm can be used to solve Sudoku [9], Boolean satisfiability problems [10], and algebraic equations [11].

In all published examples, Grover’s algorithm finds exact solutions. But what if the solution is a repeating decimal that cannot be represented exactly with any number of binary digits (bits)? We show how to use Grover’s algorithm to find approximate solutions to algebraic equations, significantly expanding the range of possible exercises. Students can design quantum circuits to solve problems approximately and then test their circuits on simulators and even real devices [12, 13].

No specialized background is required, as we introduce all necessary information. According to an instructor’s preference, this exercise can be a standalone topic, or it can be embedded in a quantum computing unit or course. In section 2, we review Boolean algebra and its application to classical arithmetic circuits. In section 3, we introduce the foundations of quantum computing. In section 4, we provide an overview of Grover’s algorithm, omitting derivations and generalizations, which are found in textbooks. In section 5, we use Grover’s algorithm to solve  $3x + 1 = 0$  to the nearest  $\frac{1}{2}$  (and to the nearest  $\frac{1}{4}$  in supplemental material). In section 6, we use Grover’s algorithm to approximately solve the transcendental equation  $2^x + (x - 1)^2 - 4 = 0$ . The same principles can be used to approximately solve countless other problems.

## 2. Boolean algebra and classical arithmetic circuits

Classical digital circuits consist of logic gates, whose inputs and outputs are bits, each of which is 0 or 1. A variable that can be only 0 or 1 is called a Boolean variable, and the algebra of Boolean variables is Boolean algebra.

Each logic gate performs a Boolean operation. The NOT gate outputs the opposite of its input. Representing the NOT operation by an overline,  $\bar{0} = 1$  and  $\bar{1} = 0$ . The AND gate outputs 1 only if both of its inputs are 1; the output is the binary multiplication of the input bits. Using algebraic multiplication to represent AND,  $A \cdot B$  is 1 if and only if  $A$  and  $B$  are both 1. The OR gate outputs 1 if at least one of its inputs is 1. Representing OR with an addition symbol,  $A + B$  is 1 if either or both variables are 1. (We infer from context whether ‘+’ means OR or addition.) Finally, the exclusive OR gate, represented by  $\oplus$ , outputs 1 if exactly one of its two inputs is 1. More generally, if the exclusive OR gate has more than two inputs, the output is 1 if an odd number of inputs are 1.

In quantum computing, the exclusive OR ( $\oplus$ ) is easier to implement than the OR (+). Since  $A + B$  is 1 when  $A \oplus B$  is 1 or when  $A \cdot B$  is 1,

$$A + B = (A \oplus B) + A \cdot B. \quad (1)$$

But since  $A \oplus B$  and  $A \cdot B$  cannot both be 1 at the same time, we can replace + with  $\oplus$ :

$$A + B = A \oplus B \oplus A \cdot B. \quad (2)$$

The order of operations is AND ( $\cdot$ ) before exclusive OR ( $\oplus$ ).

**Table 1.** Sum of two bits. The carry bit is the more significant bit of the sum, and the nominal ‘sum’ bit is the less significant bit of the sum.

| $A_0$ | $A_1$ | Carry = $A_1 \cdot A_0$ | Sum = $A_0 \oplus A_1$ |
|-------|-------|-------------------------|------------------------|
| 0     | 0     | 0                       | 0                      |
| 0     | 1     | 0                       | 1                      |
| 1     | 0     | 0                       | 1                      |
| 1     | 1     | 1                       | 0                      |

Since digital circuits consist of 0’s and 1’s, we need to represent numbers in binary. For example, the binary representation of  $13 = 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1$  is 1101. We infer from context whether a number is in binary or base ten. To understand digital adder circuits, we need to know how to add binary numbers. To sum the binary numbers 1011 and 11, we sum one column at a time, carrying the 1 in the case of  $1 + 1 = 10$  and  $1 + 1 + 1 = 11$ :

$$\begin{array}{r}
 11 \\
 1011 \\
 + \quad 11 \\
 \hline
 1110
 \end{array}$$

If we need to sum two Boolean variables  $A_0$  and  $A_1$ , there are four possible combinations of values, shown in table 1. The first row calculates  $0 + 0 = 00$ , the second row calculates  $0 + 1 = 01$ , and so on. We see that the carry bit is 1 only if both  $A_0$  and  $A_1$  are 1, so the carry bit is given by the AND operation (which is the same as binary multiplication):  $A_1 \cdot A_0$ . The sum bit is 1 only if  $A_0$  or  $A_1$  is 1, but not both, so the sum bit is given by the exclusive OR:  $A_0 \oplus A_1$ .

If we need to sum larger numbers, the process is more complicated. Rather than designing a general-purpose adder, we will consider a special case to illustrate the process. Suppose we have a two-bit number  $A_1A_0$ , and we want to add 1 to it:

|   | $A_1 \cdot A_0$ | $A_0$            | $A_1$ | $A_0$            |
|---|-----------------|------------------|-------|------------------|
| + |                 |                  |       | 1                |
|   | $A_1 \cdot A_0$ | $A_0 \oplus A_1$ |       | $\overline{A_0}$ |

In the rightmost column, we add 1 to  $A_0$ . If  $A_0 = 0$ , the sum is 1, and if  $A_0 = 1$ , the sum is 10, where the 1 is carried. In both cases, the rightmost sum bit is  $\overline{A_0}$ .

In the middle column, we have the carry bit, which equals  $A_0$  because there is a carry only if  $A_0 = 1$ . So we sum  $A_0$  and  $A_1$ , using table 1 to determine the sum bit and the carry bit.

To solve algebraic equations, we will need to represent fractions and negative numbers. We can use a decimal point in a binary number so that the first bit to the right of it represents  $\frac{1}{2}$ , the next bit to the right represents  $\frac{1}{4}$ , and so on. For example,  $10.01 = 2 + \frac{1}{4}$ .

A standard way to represent negative numbers is called two’s complement representation, in which the most significant bit represents the negative of its usual value. For example,  $1001 = -8 + 1 = -7$ . Subtraction can be performed by addition of a negative number. For example,  $2 - 7 = 2 + (-7) = 0010 + 1001 = 1011 = -5$ .

Two’s complement can be combined with a decimal point so that  $10.01 = -2 + \frac{1}{4} = -1\frac{3}{4}$ . In both classical and quantum circuits, the decimal point exists only in the mind of the user, so

**Table 2.** Decimal values in eight 3-bit representations. xxx indicates no fractional bits, xx.x indicates one fractional bit, x.xx indicates two fractional bits, and 0.xxx indicates three fractional bits.

|                   | 000 | 001           | 010           | 011            | 100            | 101             | 110            | 111            |
|-------------------|-----|---------------|---------------|----------------|----------------|-----------------|----------------|----------------|
| unsigned, xxx     | 0   | 1             | 2             | 3              | 4              | 5               | 6              | 7              |
| unsigned, xx.x    | 0   | $\frac{1}{2}$ | 1             | $1\frac{1}{2}$ | 2              | $2\frac{1}{2}$  | 3              | $3\frac{1}{2}$ |
| unsigned, x.xx    | 0   | $\frac{1}{4}$ | $\frac{1}{2}$ | $\frac{3}{4}$  | 1              | $1\frac{1}{4}$  | $1\frac{1}{2}$ | $1\frac{3}{4}$ |
| unsigned, 0.xxx   | 0   | $\frac{1}{8}$ | $\frac{1}{4}$ | $\frac{3}{8}$  | $\frac{1}{2}$  | $\frac{5}{8}$   | $\frac{3}{4}$  | $\frac{7}{8}$  |
| two's comp, xxx   | 0   | 1             | 2             | 3              | -4             | -3              | -2             | -1             |
| two's comp, xx.x  | 0   | $\frac{1}{2}$ | 1             | $1\frac{1}{2}$ | -2             | $-1\frac{1}{2}$ | -1             | $-\frac{1}{2}$ |
| two's comp, x.xx  | 0   | $\frac{1}{4}$ | $\frac{1}{2}$ | $\frac{3}{4}$  | -1             | $-\frac{3}{4}$  | $-\frac{1}{2}$ | $-\frac{1}{4}$ |
| two's comp, 0.xxx | 0   | $\frac{1}{8}$ | $\frac{1}{4}$ | $\frac{3}{8}$  | $-\frac{1}{2}$ | $-\frac{3}{8}$  | $-\frac{1}{4}$ | $-\frac{1}{8}$ |

1001 can represent 9 (unsigned binary),  $-7$  (two's complement),  $2\frac{1}{4}$  (unsigned binary with two fractional bits), or  $-1\frac{3}{4}$  (two's complement with two fractional bits). Additional values are possible if the decimal point is shifted. Further examples, with 3-bit numbers, are in table 2.

### 3. Foundations of quantum computing

While a classical bit is either 0 or 1, a quantum bit (qubit) can be in a superposition,  $c_0|0\rangle + c_1|1\rangle$ , where  $c_0$  and  $c_1$  are called probability amplitudes, and  $|0\rangle$  and  $|1\rangle$  are called computational basis vectors (or states). When a qubit is measured (in the computational basis), the result is a classical bit. The probability of measuring 0 is  $|c_0|^2$ , and the probability of measuring 1 is  $|c_1|^2$ . Since probabilities sum to 1,

$$|c_0|^2 + |c_1|^2 = 1. \quad (3)$$

The general state of two qubits is the superposition  $c_{00}|0\rangle|0\rangle + c_{01}|0\rangle|1\rangle + c_{10}|1\rangle|0\rangle + c_{11}|1\rangle|1\rangle$ , where  $|c_{00}|^2$  is the probability of measuring 00, and so on. The notation may be simplified to  $c_{00}|00\rangle + c_{01}|01\rangle + c_{10}|10\rangle + c_{11}|11\rangle$ , and further simplified by converting binary numbers to base ten:  $c_0|0\rangle + c_1|1\rangle + c_2|2\rangle + c_3|3\rangle$ . Since the four measurement probabilities sum to 1,

$$\sum_{j=0}^3 |c_j|^2 = 1. \quad (4)$$

The four basis states  $|j\rangle$  form an orthonormal basis, which means that the inner product of two basis states, written  $\langle i|j\rangle$ , is 1 if  $i=j$  and 0 otherwise.

Qubits are manipulated with operators called quantum gates. The quantum NOT gate,  $X$ , is analogous to the classical NOT:

$$X|0\rangle = |1\rangle \quad (5)$$

and

$$X|1\rangle = |0\rangle. \quad (6)$$

The  $Z$  gate has no effect on  $|0\rangle$  but multiplies  $|1\rangle$  by  $-1$ :

$$Z|1\rangle = -|1\rangle. \quad (7)$$

Our final single-qubit gate is the very useful Hadamard gate,  $H$ :

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle, \quad (8)$$

and

$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle, \quad (9)$$

where  $|+\rangle$  and  $|-\rangle$  are defined for convenience. We note for later use that

$$X|-\rangle = X\left[\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\right] = \frac{1}{\sqrt{2}}(X|0\rangle - X|1\rangle) = \frac{1}{\sqrt{2}}(|1\rangle - |0\rangle) = -|-\rangle. \quad (10)$$

If  $H$  is applied to each of two qubits initially in state  $|0\rangle$ , the result is

$$H|0\rangle H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \frac{1}{2}(|0\rangle|0\rangle + |0\rangle|1\rangle + |1\rangle|0\rangle + |1\rangle|1\rangle), \quad (11)$$

using ordinary binomial multiplication though taking care to distinguish  $|0\rangle|1\rangle$  from  $|1\rangle|0\rangle$ . Converting binary to base 10 (i.e.  $|1\rangle|1\rangle = |11\rangle = |3\rangle$ ),

$$H|0\rangle H|0\rangle = \frac{1}{2}(|0\rangle + |1\rangle + |2\rangle + |3\rangle) = \frac{1}{2} \sum_{j=0}^3 |j\rangle = |S\rangle, \quad (12)$$

defining  $|S\rangle$  as the equally weighted superposition of all basis states. We note for later use that in this case of two qubits,

$$\langle i|S\rangle = \frac{1}{2} \quad (13)$$

for all four basis states  $|i\rangle$ . Equation (12) generalizes to  $n$  qubits:

$$|S\rangle = \frac{1}{\sqrt{2^n}} \sum_{j=0}^{2^n-1} |j\rangle. \quad (14)$$

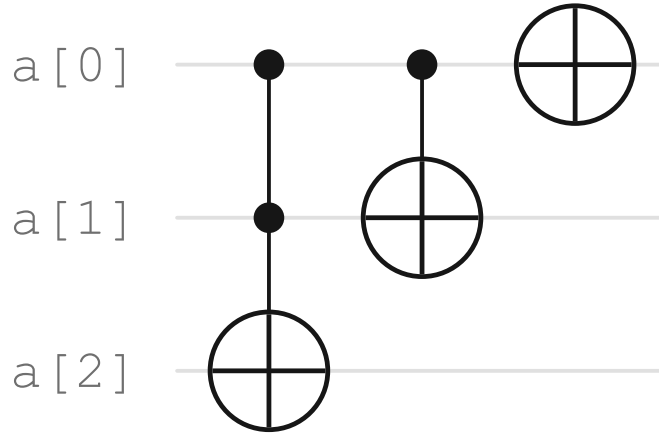
We now will introduce quantum gates that act on two or more qubits. The controlled NOT gate, CNOT, applies a NOT to the target qubit only if the control qubit is  $|1\rangle$ . Writing the control qubit first,  $\text{CNOT}|0\rangle|0\rangle = |0\rangle|0\rangle$ ,  $\text{CNOT}|0\rangle|1\rangle = |0\rangle|1\rangle$ ,  $\text{CNOT}|1\rangle|0\rangle = |1\rangle|1\rangle$ , and  $\text{CNOT}|1\rangle|1\rangle = |1\rangle|0\rangle$ . All four cases are given by the general expression

$$\text{CNOT}|i\rangle|j\rangle = |i\rangle|i \oplus j\rangle, \quad (15)$$

as students can confirm by plugging in the four combinations of values of  $i$  and  $j$ . Another controlled gate is the controlled  $Z$ , which applies a  $Z$  to the target when the control is  $|1\rangle$ .

The Toffoli gate has two control qubits and one target. A NOT is applied to the target only if both controls are  $|1\rangle$ :  $\text{Toffoli}|i\rangle|j\rangle|k\rangle = |i\rangle|j\rangle|i \cdot j \oplus k\rangle$ . (Students can test all eight combinations of values of  $i$ ,  $j$ , and  $k$  to confirm that the rightmost qubit becomes  $|\bar{k}\rangle$  if and only if  $|i\rangle$  and  $|j\rangle$  are both  $|1\rangle$ .) If the target is initially  $|0\rangle$ , then

$$\text{Toffoli}|i\rangle|j\rangle|0\rangle = |i\rangle|j\rangle|i \cdot j\rangle \quad (16)$$



**Figure 1.** Addition of 1. The target symbol is a NOT, and the dots are controls.

using the identity  $x \oplus 0 = x$ . (This may be proved simply by testing both values of  $x$ ). So we have a Toffoli gate to implement the AND operation, a CNOT gate to implement the exclusive OR operation, and the  $X$  gate to implement the NOT operation.

In quantum circuit diagrams, each qubit is represented by a horizontal line. We have to decide whether  $|i\rangle|j\rangle$  means that  $|i\rangle$  is the top qubit or the bottom qubit. Both conventions are in widespread use. We will follow the convention of IBM Quantum, in which the bottom qubit in a circuit diagram is written on the left in a mathematical expression:  $|i\rangle|j\rangle$  means that  $|i\rangle$  is the bottom qubit. So the most significant bit in a binary number will be represented by the bottom qubit.

Since the target of a CNOT or Toffoli is just as likely to be the bottom qubit as the top, we will rewrite equations (15) and (16) with the target qubits on the left (the earlier equations apply when the target is on the top):

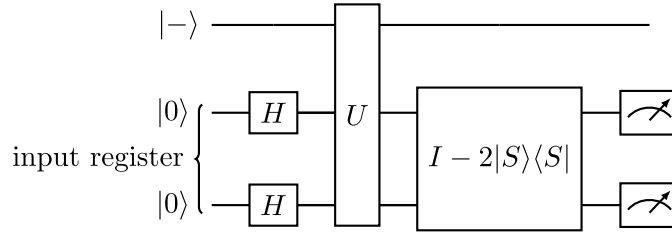
$$\text{CNOT}|j\rangle|i\rangle = |i \oplus j\rangle|i\rangle, \quad (\text{target on bottom}) \quad (17)$$

and

$$\text{Toffoli}|0\rangle|j\rangle|i\rangle = |i \cdot j\rangle|j\rangle|i\rangle. \quad (\text{target on bottom}) \quad (18)$$

We now have all the quantum gates needed to perform arithmetic. We know from the previous section that if 1 is added to the two-bit number  $A_1A_0$ , the three bits of the sum are given by  $A_1 \cdot A_0$ ,  $A_0 \oplus A_1$ , and  $\overline{A_0}$ . We want to design the quantum circuit that performs this operation, transforming  $|0\rangle|A_1\rangle|A_0\rangle$  into  $|A_1 \cdot A_0\rangle|A_0 \oplus A_1\rangle|\overline{A_0}\rangle$ . This is shown in figure 1, created using IBM Quantum, and explained below.  $A_0$  appears as  $a[0]$ . The initial state of each qubit is  $|0\rangle$  by default but can be changed to  $|1\rangle$  with a NOT gate.

Quantum circuit diagrams are read left to right. The first gate is a Toffoli, which turns  $|A_2\rangle = |0\rangle$  into  $|A_1 \cdot A_0\rangle$ , according to equation (18). The next gate is a CNOT, which turns  $|A_1\rangle$  into  $|A_0 \oplus A_1\rangle$ , according to equation (17). The final gate is a NOT, which turns  $|A_0\rangle$  into  $|\overline{A_0}\rangle$ . Students can quickly construct this circuit with a graphical interface (the ‘Composer’) and immediately see the simulated sum. NOT gates placed on the left on  $|A_0\rangle$  and  $|A_1\rangle$  can be used to create all four possible combinations of initial values.



**Figure 2.** Grover's algorithm with two qubits in the input register. The bottom two qubits represent four possible solutions: 00, 01, 10, and 11.  $U$  is a special operator that depends on the problem being solved. The symbols on the far right represent measurements.

#### 4. Overview of Grover's search algorithm

The goal of Grover's search algorithm is to have a high probability of measuring one 'good' state,  $|G\rangle$ , at the end of the quantum circuit.  $G$  is the solution to any chosen problem that can be solved by trial and error. In our case,  $G$  will be the best approximate solution to an equation. Grover's algorithm solves problems by simultaneously testing each of  $2^n$  possible solutions, where  $n$  is the number of qubits used to represent possible solutions. These qubits are collectively called the input register. (A register is a group of qubits.) The circuit for Grover's algorithm with  $n = 2$  is shown in figure 2. We present Grover's algorithm without derivations or generalizations, as these are found in textbooks [7, 13].

$U$  is a special operator called an oracle. The effect of  $U$  in Grover's algorithm is to multiply  $|G\rangle$  by  $-1$  without affecting other basis states:

$$U|j\rangle|-\rangle = \begin{cases} |j\rangle|-\rangle, & j \neq G, \\ -|j\rangle|-\rangle, & j = G. \end{cases} \quad (19)$$

$|j\rangle$  represents a basis state of the input register, the bottom two qubits. The top qubit is included for the convenience of using equation (10) to generate a factor of  $-1$ . In the next section, we will see how to design  $U$  to apply a NOT to the top qubit when the input register is  $|G\rangle$ . We assume there is only one good state (in our case, one approximate solution within a chosen error tolerance), though generalization is possible.

The second special operator in Grover's algorithm is  $I - 2|S\rangle\langle S|$ , where  $I$  is the identity operation (it leaves unchanged any state), and  $|S\rangle$  is the superposition of basis states defined in equation (12). This operator is usually given as  $2|S\rangle\langle S| - I$ , but the circuit design is simpler when this operator is multiplied by  $-1$ . A factor of  $-1$  applied to all states is called a global phase factor, and it has no physical effect and can be ignored. Textbooks [13] explain that  $I - 2|S\rangle\langle S|$  can be constructed from elementary gates as shown in figure 3.

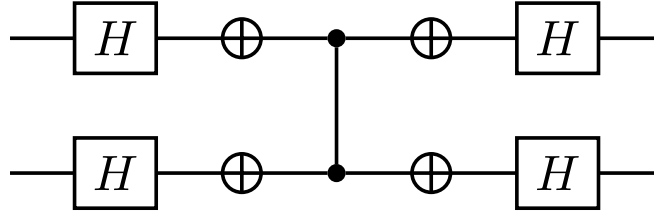
When the input register has more than two qubits, the  $U$  and  $I - 2|S\rangle\langle S|$  operators have to be applied in alternation more than once. Textbooks [7, 13, 14] show that the optimal number of times to apply these operators is

$$t = \frac{\pi}{4} \sqrt{2^n} - \frac{1}{2}, \quad (20)$$

where  $n$  is the number of qubits in the input register, and  $t$  is rounded to the nearest integer.

We will briefly demonstrate that the circuit in figure 2 yields the good solution  $G$ . Suppose  $G = 11$  in binary. The initial state is  $|0\rangle|0\rangle|-\rangle$ . After the  $H$  gates, the state is  $\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)|-\rangle$ , using equation (12) and converting to binary. The  $U$  gate





**Figure 3.**  $I - 2|S\rangle\langle S|$  for two qubits. The vertical line with two dots is a controlled Z and is constructed in the IBM Quantum Composer by dropping a control ‘dot’ onto a Z gate.

multiplies only the good state,  $|11\rangle$ , by  $-1$ , resulting in  $\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle)|-\rangle$ .  $U$  has acted simultaneously on all four possible solution states and flagged the correct solution with the factor of  $-1$ . This simultaneous process is made possible by quantum superposition and confers a quantum speedup compared with a brute-force search on a classical computer.

Finally,  $I - 2|S\rangle\langle S|$  is applied to the input register.  $I$  and  $-2|S\rangle\langle S|$  act separately:

$$\begin{aligned} & (I - 2|S\rangle\langle S|)\left(\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle)\right) \\ &= \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle) - |S\rangle(\langle S|00\rangle + \langle S|01\rangle + \langle S|10\rangle - \langle S|11\rangle). \end{aligned} \quad (21)$$

Each inner product is  $\frac{1}{2}$ , according to equation (13), so the value in the rightmost parentheses is  $\frac{1}{2} + \frac{1}{2} + \frac{1}{2} - \frac{1}{2} = 1$ . The expression simplifies to

$$\frac{1}{2}(|01\rangle + |01\rangle + |10\rangle - |11\rangle) - |S\rangle. \quad (22)$$

Since  $|S\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$  from equation (12), equation (22) simplifies further to  $-|11\rangle$ . Thus the probability of measuring the good result, 11, is  $(-1)^2 = 1$ . When there are more than two qubits in the input register, the probability of the good result is less than 100% but at least 94% [14].

## 5. Solving $3x + 1 = 0$ with Grover’s algorithm

We will now use Grover’s algorithm to obtain an approximate solution to the equation  $3x + 1 = 0$ . As the solution is  $-\frac{1}{3}$ , it cannot be represented exactly with any number of bits. Thus, a difference between the exact solution and the circuit output is an unavoidable error. This error can be reduced indefinitely by increasing the number of bits of precision, although the quantum circuit size and complexity increase correspondingly.

We will make many simplifying assumptions, analogous to how we ignore friction and air resistance in introductory mechanics: We need to learn how to solve contrived problems before we can solve realistic ones.

We first need to decide how many bits of precision we want for  $x$ . For simplicity, we will choose two bits, and we will use  $j$  instead of  $x$  because IBM Quantum does not allow a register to be named  $x$ , a letter reserved for the NOT gate.

Out of our two bits, we need to decide how many are fractional bits, and whether to use two’s complement or unsigned binary. We choose to use two’s complement with one fractional bit. So the four possible values of  $j$  are  $00 \Rightarrow 0$ ,  $01 \Rightarrow \frac{1}{2}$ ,  $10 \Rightarrow -1$ , and  $11 \Rightarrow -\frac{1}{2}$ . Grover’s algorithm will effectively test these four values to determine which is the best

**Table 3.** Multiplication by 3. The first column gives the two's complement representation, with one fractional bit, of the base-10 number in the second column. The third column multiplies the second column by 3, and the fourth column is the two's complement representation, with one fractional bit, of the result.

| $j$ (2's comp, $j_1 \cdot j_0$ ) | $j$ (base 10)  | $3j$ (base 10) | $3j$ (2's comp, $a_3a_2a_1 \cdot a_0$ ) |
|----------------------------------|----------------|----------------|-----------------------------------------|
| 00                               | 0              | 0              | 0000                                    |
| 01                               | $\frac{1}{2}$  | $\frac{3}{2}$  | 0011                                    |
| 10                               | -1             | -3             | 1010                                    |
| 11                               | $-\frac{1}{2}$ | $-\frac{3}{2}$ | 1101                                    |

solution to  $3j + 1 = 0$ . In the supplementary material, we show how to increase the precision of the solution; both the precision and the range of possible solutions can be increased indefinitely, at the price of increased circuit complexity.

We will need a register of auxiliary qubits (called ancilla qubits),  $|a\rangle$ , to compute  $a = 3j + 1$ . We will use four bits for  $a$ , choosing two's complement with one fractional bit. So there are 16 possible values of  $a$ , ranging from  $1000 \Rightarrow -4$  to  $0111 \Rightarrow 3.5$ .

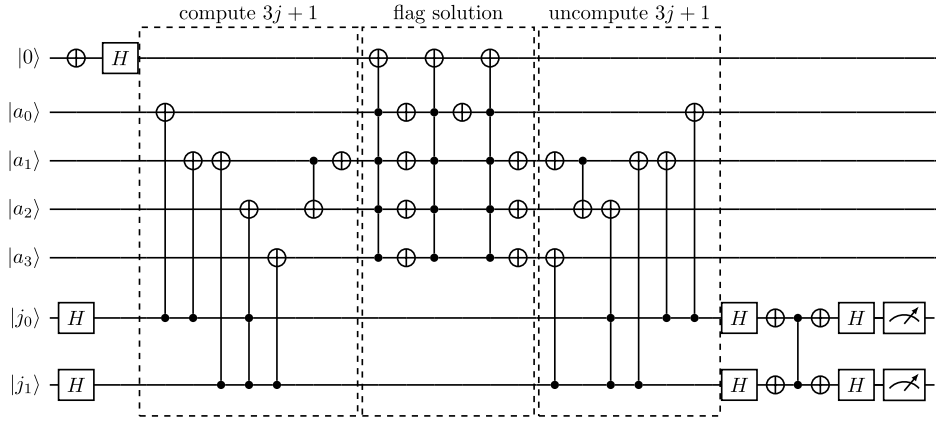
It is possible to design quantum circuits to perform any function that classical digital circuits can perform, like arithmetic. A quantum multiplier is complicated, but a general-purpose quantum adder is achievable [13]. However, for simplicity, we design a quantum circuit that first performs multiplication by 3, and then addition of 1, without considering more general addition. To solve any other problem, a different circuit must be designed by hand. This is useful for instructional purposes. However, to solve larger problems requiring dozens of qubits, it is not practical to hand-design at the level of individual qubits and gates. Scalable, general-purpose subroutines must be implemented instead. The ripple-carry adder, for example, can be scaled to sum any two numbers of arbitrary size [13]. Alternative adder circuits use fewer qubits but are conceptually more complicated [15].

To perform multiplication by 3, we compute  $3j$  for each of value of  $j$ , shown in table 3. We need to determine  $a_2$ ,  $a_1$ , and  $a_0$  in terms of  $j_1$  and  $j_0$ . By comparing the first and last columns of the table, we see that  $a_0$  is the same as  $j_0$ , so  $a_0 = j_0$ .  $a_1$  is 1 when the two bits of  $j$  differ from each other, so  $a_1 = j_1 \oplus j_0$ .  $a_2$  is 1 when both  $j_1$  and  $j_0$  are 1, so  $a_2 = j_1 \cdot j_0$ . Finally,  $a_3 = j_1$ .

Next we need to create the quantum circuit to establish the necessary values of  $|a\rangle$ . The multiplication by 3 is implemented by the first five gates in the 'compute  $3j + 1$ ' section in figure 4. The first CNOT gate changes  $|a_0\rangle$  to  $|1\rangle$  if  $|j_0\rangle$  is  $|1\rangle$ , so it sets  $|a_0\rangle = |j_0\rangle$ . The next CNOT makes  $|a_1\rangle = |j_0\rangle$ , and the third CNOT makes  $|a_1\rangle = |j_1 \oplus j_0\rangle$ , using equation (15). The Toffoli makes  $|a_2\rangle = |j_1 \cdot j_0\rangle$ , using equation (16). The final CNOT sets  $|a_3\rangle = |j_1\rangle$ . Now all four qubits of  $|a\rangle$  have the required value from table 3.

The next step is to add 1. We use the circuit introduced in figure 1, with small modifications: There is now one fractional bit, so we shift down one qubit; addition of 1 does not affect the fractional bit in  $|a_0\rangle$ . Also, we omit the Toffoli gate because we see in table 3 that  $a_1$  and  $a_2$  are never both 1 at the same time. The two required gates to implement addition of 1 are the final two gates in the 'compute  $3j + 1$ ' section in figure 4.

Now we will put these circuit fragments in context. The initial state of the seven qubits in figure 4 is  $|0000000\rangle$  by default. Just before the 'compute  $3j + 1$ ' section, the state is  $\frac{1}{2} \sum |j\rangle |0000\rangle |-\rangle$  because the bottom two Hadamard gates create the superposition in the input register  $|j\rangle$ , and the gates on the top qubit create the state  $|-\rangle$ . The sum is over the four possible two-bit values of  $j$ : 00, 01, 10, and 11. At the end of the 'compute  $3j + 1$ ' section,



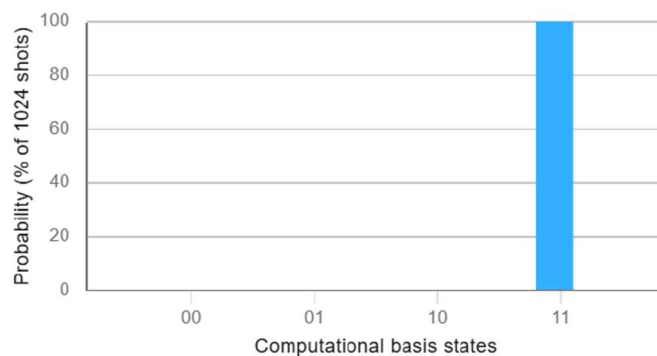
**Figure 4.** Grover’s algorithm used to find an approximate solution to  $3j + 1$ . The default initial state of every qubit is  $|0\rangle$ . The three boxed sections combine to form the oracle operator  $U$ .

the state is  $\frac{1}{2} \sum |j\rangle |3j + 1\rangle |-\rangle$ . Let us explicitly write out the four terms in the summation, using the base-10 values in table 3 (and adding 1 to change  $3j$  to  $3j + 1$ ):  $\frac{1}{2}(|0\rangle|1\rangle + |\frac{1}{2}\rangle|\frac{5}{2}\rangle + |-1\rangle|-2\rangle + |-\frac{1}{2}\rangle|-\frac{1}{2}\rangle)|-\rangle$ . The  $|j\rangle$  register is said to be entangled with the  $|a\rangle$  register because each value in one register corresponds with a specific value in the other.

Next we have gates that apply a NOT to the top qubit whenever the  $|j\rangle$  register holds the best approximate solution to  $3j + 1 = 0$ , which is when the corresponding  $a$  differs from 0 by  $\frac{1}{2}$  at most. A NOT applied to the top qubit will change  $|-\rangle$  to  $-|-\rangle$  according to equation (10). This marks the best approximation as the good state that we want to measure. For simplicity, we assume that only one value of  $j$  produces  $3j + 1$  within  $\frac{1}{2}$  of 0. Since we have one fractional bit of precision, there are three possible values of  $3j + 1$  within  $\frac{1}{2}$  of 0:  $-\frac{1}{2}$ , 0, and  $\frac{1}{2}$ . These correspond with  $a$  values of 1111, 0000, and 0001, using two’s complement with one fractional bit. (To review,  $1111 = -4 + 2 + 1 + \frac{1}{2} = -\frac{1}{2}$ .)

The first gate in the ‘flag solution’ section in figure 4 is a quadruply controlled NOT, which applies a NOT to the top qubit when  $|a\rangle = |1111\rangle$ . Then we have four NOT gates which change  $|0000\rangle$  to  $|1111\rangle$ , so the second quadruply controlled NOT gate acts if  $|a\rangle = |0000\rangle$  going into the ‘flag solution’ section. The final quadruply controlled NOT gate has controls of  $|1111\rangle$  if  $|a\rangle = |0001\rangle$  going into the ‘flag solution’ section. The next three NOT gates restore  $|a\rangle$  to its value going into the ‘flag solution’ section. The overall effect of the ‘flag solution’ section is to attach a factor of  $-1$  to the state that solves  $3j + 1 = 0$  within an error tolerance of  $\frac{1}{2}$ . Specifically, the state after the ‘flag solution’ section is  $\frac{1}{2}(|0\rangle|1\rangle + |\frac{1}{2}\rangle|\frac{5}{2}\rangle + |-1\rangle|-2\rangle - |-\frac{1}{2}\rangle|-\frac{1}{2}\rangle)|-\rangle$ , using base 10: The fourth term has been multiplied by  $-1$  because here  $|a\rangle = |-\frac{1}{2}\rangle = |1111\rangle$ . We need to fluidly convert between base 10 and the binary representation: We perform  $3j + 1$  in base 10, and we use the binary representation to specify the states of individual qubits.

The gates in the ‘undo  $3j + 1$ ’ section reverse the  $+1$  and  $\times 3$  operations, restoring  $|a\rangle$  to  $|0\rangle$  by applying the gates in the opposite order. This is an essential step called uncomputing, which removes the entanglement between the  $|j\rangle$  and  $|a\rangle$  registers, while retaining the factor of  $-1$  on the fourth term. The state is now  $\frac{1}{2}(|0\rangle + |\frac{1}{2}\rangle + |-1\rangle - |-\frac{1}{2}\rangle)|00\rangle|-\rangle$ . The  $|j\rangle$



**Figure 5.** Simulated results for the circuit that solves  $3j + 1 = 0$ , created using IBM Quantum.

register must be free of entanglement when it encounters the  $I - 2|S\rangle\langle S|$  operator at the end of the circuit. The  $I - 2|S\rangle\langle S|$  operator works correctly only when the  $|j\rangle$  register is effectively isolated from the rest of the circuit. The necessity of uncomputing is not obvious, but students can easily simulate in IBM Quantum what happens if the uncomputing stage is eliminated: Instead of outputting the single best approximate solution, the circuit outputs any of the four possible values of the  $|j\rangle$  register with equal probability.

Now we proceed to the final operator,  $I - 2|S\rangle\langle S|$ . Just before this operator, the state is  $\frac{1}{2}(|0\rangle + |\frac{1}{2}\rangle + |-1\rangle - |-\frac{1}{2}\rangle)|00\rangle|-\rangle$ , or in the binary representation,  $\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle)|00\rangle|-\rangle$ . The  $|j\rangle$  register is in the exact superposition that  $I - 2|S\rangle\langle S|$  acted on in equation (21). As shown previously, this operator converts the  $|j\rangle$  register to its final state,  $-|11\rangle$ , where 11 now represents  $-\frac{1}{2}$ .

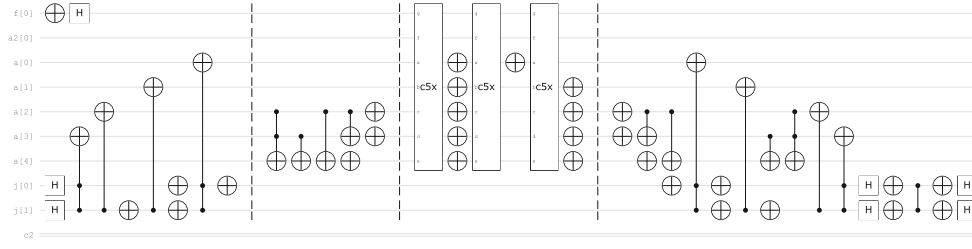
We constructed the circuit in figure 4 in the IBM Quantum Composer. The simulation of measurement probabilities is shown in figure 5. There is theoretically a 100% probability of 11, which represents  $j = -\frac{1}{2}$ . This indeed is the best solution to  $3j + 1 = 0$  to the nearest  $\frac{1}{2}$ . Students can think about how to modify the circuit to solve  $3j = 0$  (delete the gates that perform +1 and its inverse) and  $3j + 2 = 0$  (addition of 2 is addition of binary 10, so the gates that perform +1 and its inverse move down one qubit).

We can increase the number of qubits in the  $|j\rangle$  register to improve the precision or broaden the range of tested values, or both. There are limitless variations for students to explore. In the supplementary material, we solve  $3j + 1 = 0$  to the nearest  $\frac{1}{4}$ . The precision improves by a factor of 2 with each additional qubit in the eigenvalue register, if the tested range remains the same. Alternatively, each additional qubit doubles the tested range if the precision remains unchanged.

## 6. Solving a transcendental equation with Grover's algorithm

We now show how to use the same techniques to solve a transcendental equation, which cannot be solved analytically. We do not claim that Grover's algorithm is more efficient at solving transcendental equations than classical numerical techniques. However, solving transcendental equations could become a subroutine in a larger quantum algorithm, in the way that modular multiplication is used in Shor's factoring algorithm [16].

The equation we will solve is  $2^j + (j - 1)^2 - 4 = 0$ . We assume that one integer from 0 to 3 solves the equation with an error of 1 at most. Our input register will thus be  $|j_1 j_0\rangle$ , where  $j_1 j_0$  is now an unsigned binary integer with no fractional bits. We will have five qubits in the



**Figure 6.** Grover's algorithm used to find an approximate solution to  $2^j + (j - 1)^2 - 4 = 0$ . The top qubit is arbitrarily labeled  $f[0]$ .

**Table 4.** Calculation of  $2^j + (j - 1)^2$ . The first column gives the binary representation of the base-10 number in the second column. The third column performs the calculation, and the fourth column is the binary representation of the result. The binary numbers in this table are unsigned and do not have any fractional bits.

| $j$<br>$j$ (unsigned, $j_1j_0$ ) | $j$<br>(base 10) | $2^j + (j - 1)^2$ in base 10 | $2^j + (j - 1)^2$ as<br>$a_4a_3a_2a_1a_0$ |
|----------------------------------|------------------|------------------------------|-------------------------------------------|
| 00                               | 0                | 2                            | 00010                                     |
| 01                               | 1                | 2                            | 00010                                     |
| 10                               | 2                | 5                            | 00101                                     |
| 11                               | 3                | 12                           | 01100                                     |

ancilla register  $|a\rangle$ , where  $a_4a_3a_2a_1a_0$  is a two's complement representation with no fractional bits, so that  $a_4$  represents  $-16$ .

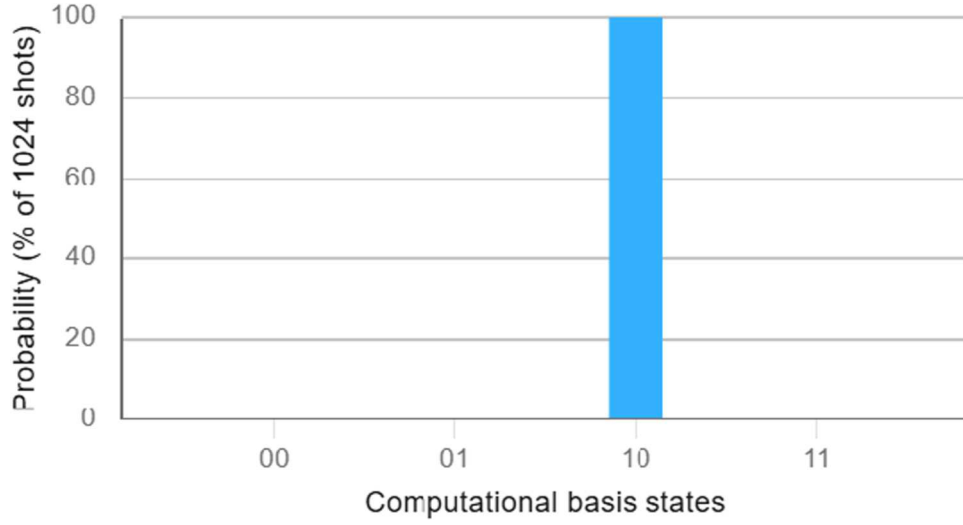
We will compute  $2^j + (j - 1)^2$  in one step. The second step, subtracting 4 (adding  $-4 = 11100$ ), would ideally be done by a generic adder. For simplicity, our circuit will subtract 4 without considering more general arithmetic. Again, our circuit is for instructional purposes and is not scalable to an arbitrarily high precision or range of trial values.

Table 4 shows the calculation of  $2^j + (j - 1)^2$ . We see that  $a_4$  is always 0.  $a_3$  is 1 when  $j_1$  and  $j_0$  are both 1, so  $a_3 = j_1 \cdot j_0$ .  $a_2$  always has the value of  $j_1$ , so  $a_2 = j_1$ .  $a_1$  is 1 when  $j_1$  is 0, so  $a_1 = \bar{j}_1$ . Finally,  $a_0$  is 1 when  $j_1$  is 1 and  $j_0$  is 0, so  $a_0 = j_1 \cdot \bar{j}_0$ .

The values of  $|a\rangle$  are set by the Toffoli and CNOT gates before the first dashed line in figure 6. The first Toffoli sets  $|a_3\rangle = |j_1 \cdot j_0\rangle$ . The first CNOT sets  $|a_2\rangle = |j_1\rangle$ . The next CNOT, with NOT gates around the control, sets  $|a_1\rangle = |\bar{j}_1\rangle$ . (The NOT before the control changes  $|j_1\rangle$  into  $|\bar{j}_1\rangle$ , and the NOT after the control restores  $|j_1\rangle$  for use in the next gate.) The Toffoli, with NOT gates around the top control, sets  $|a_0\rangle = |j_1 \cdot \bar{j}_0\rangle$ .

Next, we have to subtract 4, which is the same as adding  $-4$ , which means adding 11100, according to 5-bit two's complement, which was chosen for the  $|a\rangle$  register. We perform the long addition of 11100 and  $a_3a_2a_1a_0$  (we know from table 4 that  $a_4 = 0$ ):

|   | $a_2 + a_3$            | $a_2$                       | $a_1$            | $a_0$ |
|---|------------------------|-----------------------------|------------------|-------|
|   | 0                      | $a_3$                       | $a_2$            | $a_1$ |
| + | 1                      | 1                           | 1                | 0     |
|   | $\overline{a_2 + a_3}$ | $\overline{a_2 \oplus a_3}$ | $\overline{a_2}$ | $a_1$ |
|   |                        |                             |                  | $a_0$ |



**Figure 7.** Simulated results for the circuit that solves  $2^j + (j - 1)^2 - 4 = 0$ , created using IBM Quantum.

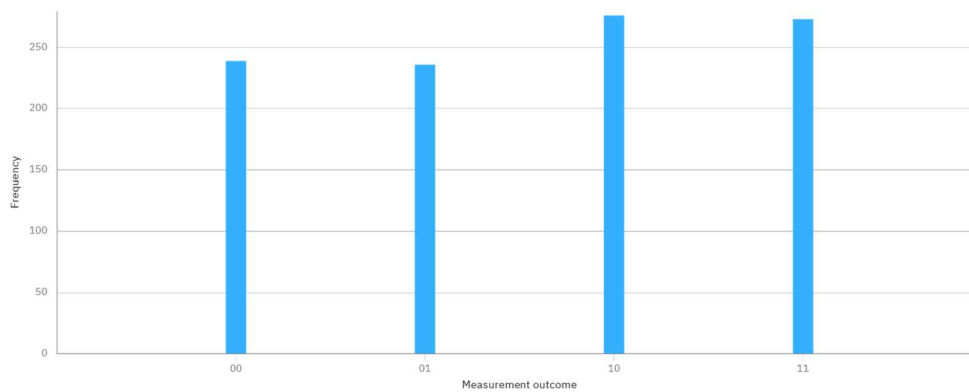
The two rightmost bits of 11100 are 0, so the addition does not affect the two least significant qubits of  $|a\rangle$ . When  $a_2$  is added to 1, the sum bit is  $\overline{a_2}$ , and the carry bit is  $a_2$ . When this carry bit is added to  $a_3$  and 1, the sum bit is  $\overline{a_2 \oplus a_3}$ , and the carry bit is  $a_3 + a_2$ , where the  $+$  here is a Boolean OR. (There is a carry if either  $a_3$  or  $a_2$  or both are 1.) When 1 is added to this carry bit, the sum bit is  $\overline{a_2 + a_3}$ . We do not need to compute a final carry bit, which would be a sixth bit; we know, from table 4, that 4 will be subtracted from either 2, 5, or 12, so the result is within the range of 5 bit two's complement. (The range is from  $10000 \Rightarrow -16$  to  $01111 \Rightarrow 15$ .)

Since the most significant qubit of the sum is  $\overline{a_3 + a_2}$ , we use equation (2) to express this as  $\overline{a_3 \oplus a_2 \oplus a_3 \cdot a_2}$ . The  $a_2 \cdot a_3$  term is added with the first Toffoli after the first dashed line in figure 6, and the individual  $a_3$  and  $a_2$  terms are added with CNOT gates. Next, the NOT gate on  $|a_4\rangle$  adds the overline. The addition of  $-4$  changes  $|a_3\rangle$  to  $\overline{a_2 \oplus a_3}$ , which is achieved by the CNOT and NOT gates targeting  $|a_3\rangle$ . Finally, the NOT on  $|a_2\rangle$  creates the sum bit  $\overline{a_2}$ .

Between the second and third dashed lines in figure 6, we apply a NOT to the top qubit whenever the value in the  $|a\rangle$  register is within 1 of 0; in other words, when  $a$  is  $11111 \Rightarrow -1$ ,  $00000$ , or  $00001$ . Since a quintuply controlled NOT is not provided by IBM Quantum, we construct it as shown in the supplemental material (figure S2). This requires an additional auxiliary qubit, labeled  $a2[0]$  in figure 6.

After the third dashed line, we reverse the order of the gates used before the second dashed line to return all ancilla qubits to  $|0\rangle$  (gates acting on different qubits can ‘slide’ past each other without any effect, which allows compression on the right side of figure 6), and then we implement  $I - 2|S\rangle\langle S|$ . The simulation in figure 7 shows a 100% chance of measuring 10, the binary representation of 2, the solution to  $2^j + (j - 1)^2 - 4 = 0$  to the nearest positive integer.

The same circuit was run 1024 times on a real quantum processor, `ibm_torino`. Results are shown in figure 8. In contrast with the (noiseless) simulated results, real results are distributed roughly evenly over the four possible values. The correct solution is impossible to



**Figure 8.** Results obtained from running the circuit that solves  $2^j + (j - 1)^2 - 4 = 0$  on a real quantum processor, `ibm_torino`. This figure was created using IBM Quantum.

infer. The circuit in figure 6 is simply too large for a noisy quantum processor to run successfully. Smaller Grover circuits do run successfully on real processors [11].

## 7. Conclusions

We have shown how to use Grover's algorithm to solve algebraic and transcendental equations approximately. This is an advanced topic that requires many skills, including binary representation of fractional and negative numbers, implementation of Boolean operations with quantum gates, and design of larger quantum gates out of an available set. These skills are broadly applicable in classical and quantum computation. We hope that this enrichment exercise empowers students to creatively design their own quantum circuits; gives them a taste of the excitement of quantum computing beyond typical textbook problems; and cultivates skills, confidence, and enthusiasm for further work in the rapidly growing field of quantum computing.

Every circuit that we present works correctly when simulated. However, when we run the circuits on real quantum processors, error strongly affects the results, making the correct solutions impossible to infer. This is not a fluke but an unavoidable feature of today's noisy intermediate-scale quantum (NISQ) circuits [17]. Students can observe the discrepancy between simulation and the performance of available quantum processors. This dramatically demonstrates the need for further progress in error mitigation and other aspects of quantum device engineering. In quantum error correction, three or more qubits are typically used to represent the state of a single ideal qubit. Assuming an error occurs in only one of the qubits, the error can be diagnosed and corrected by comparing them. Quantum error mitigation is a topic of active research [18].

## Data availability statement

All data that support the findings of this study are included within the article (and any supplementary files).

## References

- [1] Hughes C, Isaacson J, Turner J, Perry A and Sun R 2022 Teaching quantum computing to high school students *Phys. Teach.* **60** 187–9
- [2] Darienzo M, Kelly A M, Schneble D and Wei T-C 2024 Student attitudes toward quantum information science and technology in a high school outreach program *Phys. Rev. Phys. Educ. Res.* **20** 020126
- [3] Metz C 2024 *Quantum Computing Inches Closer to Reality After Another Google Breakthrough* (New York Times)
- [4] Metz C 2025 *Microsoft Says It Has Created a New State of Matter to Power Quantum Computers* (New York Times)
- [5] Singh C, Asfaw A and Levy J 2021 Preparing students to be leaders of the quantum information revolution *Phys. Today* (<https://doi.org/10.1063/PT.6.5.20210927a>)
- [6] Griffiths D J 1995 *Introduction to Quantum Mechanics* (Prentice-Hall, Inc.)
- [7] Zygelman B 2018 *A First Introduction to Quantum Computing and Information* (Springer)
- [8] Kain B 2021 Searching a quantum data base with Grover’s search algorithm *Am. J. Phys.* **89** 618–26
- [9] Qiskit textbook, <https://github.com/Qiskit/textbook/blob/main/notebooks/ch-algorithms/grover.ipynb>
- [10] Cheng S-T and Tao M-H 2007 Quantum cooperative search algorithm for 3-SAT *J. Comput. Syst. Sci.* **73** 123136
- [11] Brody J and Servedio H 2025 Solving  $x + 1 = 3$  with a quantum computer *Math. Intell.* **47** 197202
- [12] (IBM Quantum) <https://quantum.cloud.ibm.com/composer>
- [13] Wong T 2022 *Introduction to Classical and Quantum Computing* (Rooted Grove)
- [14] de Lima Marquezino F, Portugal R and Lavor C 2019 *A Primer on Quantum Computing* (Springer)
- [15] Draper T G 2000 Addition on a quantum computer <https://doi.org/10.48550/arXiv.quant-ph/0008033>
- [16] Tan X J and Gao P 2024 An efficient quantum circuit implementation of Shor’s algorithm for GPU accelerated simulation *AIP Adv.* **14** 025333
- [17] Preskill J 2018 Quantum Computing in the NISQ era and beyond *Quantum* **2** 79
- [18] Cai Z *et al* 2023 “Quantum error mitigation *Rev. Mod. Phys.* **95** 045005